

ABE 715: Computer Programming

Compendium Part 8

Advanced Case Studies & Research Methodology

Prepared for:
MSc. Irrigation Engineering and Management
MSc. Agricultural and Biosystems Engineering

Instructor: Dr. Kadeghe G. Fue
Electronics and Precision Agriculture Lab

Department of Agricultural Engineering
Sokoine University of Agriculture (SUA)

Academic Year 2025/2026

Contents

Preface **ii**

1 Case Study 1: The Autonomous Greenhouse **1**
1.1 System Overview 1
1.2 The Edge Logic (C++) 1
1.3 The Brain (Python) 2

2 Case Study 2: Drone Image Pipeline **4**
2.1 Workflow Overview 4
2.2 Batch Processing Script 4

3 Computational Research Methodology **6**
3.1 From Assignment to Thesis 6
3.2 Model Validation 6
3.2.1 The Train/Test/Validate Split 6
3.3 Version Control (Git) 6
3.4 Data Management 7

4 Professional Resources **8**
4.1 Recommended Libraries 8
4.2 Online Communities 8

Preface to Part 8

This final volume of the ABE 715 series moves beyond discrete skills to integrated systems. As Master's students, you are expected not just to write code, but to engineer solutions.

This Compendium focuses on **Capstone-level implementations** where IoT, Data Science, and Control Logic merge. It also includes a guide on **Research Methodology** for computational engineering, helping you prepare for your thesis work.

Contents:

- **Case Study 1:** The Autonomous Greenhouse (Full Stack).
- **Case Study 2:** Drone Imagery Pipeline (GIS + Vision).
- **Research Guide:** Model Validation, Reproducibility, and Git.

Chapter 1

Case Study 1: The Autonomous Greenhouse

1.1 System Overview

This case study demonstrates a "Full Stack" agricultural system. It connects a physical environment to a cloud dashboard.

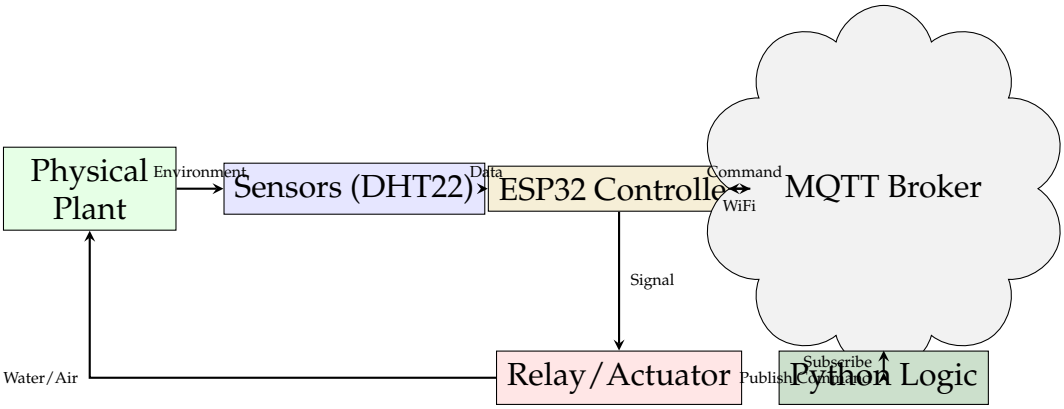


Figure 1.1: Closed-Loop Control System Architecture

1.2 The Edge Logic (C++)

The ESP32 must handle both sensing and listening for commands. This requires non-blocking code.

```
void loop() {
  client.loop(); // Keep MQTT alive

  unsigned long currentMillis = millis();

  // Read Sensors every 10 seconds (Non-blocking delay)
  if (currentMillis - previousMillis >= 10000) {
    previousMillis = currentMillis;

    float h = dht.readHumidity();
```

```
    float t = dht.readTemperature();

    // JSON Payload
    String payload = "{\"id\":1,\"temp\":" + String(t) + "}";
    client.publish("sua/greenhouse/data", payload.c_str());
  }
}

// Callback for incoming commands
void callback(char* topic, byte* message, unsigned int length) {
  String msg;
  for (int i = 0; i < length; i++) { msg += (char)message[i]; }

  if (msg == "FAN_ON") { digitalWrite(RELAY_PIN, HIGH); }
  if (msg == "FAN_OFF") { digitalWrite(RELAY_PIN, LOW); }
}
```

Listing 1.1: Non-blocking ESP32 Loop

1.3 The Brain (Python)

The Python script acts as the "Digital Twin," deciding when to actuate based on complex logic (e.g., VPD).

```
import math
import paho.mqtt.client as mqtt
import json

def calculate_vpd(temp, humidity):
    """Calculates VPD in kPa"""
    es = 0.6108 * math.exp((17.27 * temp) / (temp + 237.3))
    ea = es * (humidity / 100)
    return es - ea

def on_message(client, userdata, msg):
    payload = json.loads(msg.payload.decode())
    temp = payload['temp']
    hum = payload['hum']

    vpd = calculate_vpd(temp, hum)
    print(f"Current VPD: {vpd:.2f} kPa")

    # Logic: Optimal VPD is 0.8 - 1.2 kPa
    if vpd > 1.2:
        print("Air too dry/hot. Turning Misters ON.")
        client.publish("sua/greenhouse/cmd", "MIST_ON")
    elif vpd < 0.5:
        print("Air too humid. Turning Fans ON.")
        client.publish("sua/greenhouse/cmd", "FAN_ON")
```

Listing 1.2: Vapor Pressure Deficit (VPD) Controller

Chapter 2

Case Study 2: Drone Image Pipeline

2.1 Workflow Overview

Processing drone imagery involves handling gigabytes of data. Automation is essential.

1. **Ingest:** Read all TIF files in a directory.
2. **Process:** Calculate Vegetation Indices (NDVI, NDRE).
3. **Analyze:** Threshold to find "Stressed" areas.
4. **Output:** Generate a Shapefile for the tractor's GPS.

2.2 Batch Processing Script

This script uses `rasterio` and `glob` to process multiple images.

```
import rasterio
import numpy as np
import glob
import os

# Get list of all TIF files
image_files = glob.glob("raw_data/*.tif")

for file in image_files:
    with rasterio.open(file) as src:
        # Assuming Band 4=Red, Band 8=NIR (Sentinel/
   Multispectral)
        red = src.read(1).astype('float32')
        nir = src.read(2).astype('float32')

        # Calculate NDVI (Handle division by zero)
        ndvi = np.where(
            (nir + red) == 0.,
            0,
            (nir - red) / (nir + red)
```

```
        )

        # Save Result
        profile = src.profile
        profile.update(dtype=rasterio.float32, count=1)

        out_name = "processed/" + os.path.basename(file)
        with rasterio.open(out_name, 'w', **profile) as dst:
            dst.write(ndvi, 1)

    print(f"Processed {file}")
```

Listing 2.1: Batch NDVI Calculation

Chapter 3

Computational Research Methodology

3.1 From Assignment to Thesis

Research code differs from homework code. It must be **Reproducible**, **Robust**, and **Validated**.

3.2 Model Validation

When developing a simulation (e.g., Crop Yield Prediction), you must prove it works.

3.2.1 The Train/Test/Validate Split

Never test your model on the same data you used to train it.

- **Training Set (70%):** Used to fit the parameters ($y = mx + c$).
- **Testing Set (30%):** Used to evaluate performance (RMSE, R^2).

```python
from sklearn.metrics import mean_squared_error, r2_score

y_true = [2.5, 3.0, 4.2, 5.1] # Actual Field Data
y_pred = [2.4, 3.1, 4.0, 4.9] # Model Output

rmse = mean_squared_error(y_true, y_pred, squared=False)
r2 = r2_score(y_true, y_pred)

print(f"Model Error (RMSE): {rmse:.2f} tons/ha")
print(f"Model Fit (R2): {r2:.2f}")
```

Listing 3.1: Validation Metrics

3.3 Version Control (Git)

In a collaborative research lab, emailing code (e.g., `model_final_v3_REAL.py`) is unacceptable. We use **Git**.

Command	Action
`git init`	Start tracking a folder.
`git add .`	Stage changes.
`git commit -m "Fixed bug"`	Save a snapshot of the code history.
`git push`	Upload to GitHub/GitLab.

Table 3.1: Basic Git Commands

3.4 Data Management

- **Raw Data is Sacred:** Never modify the original CSV/Excel file. Always load it, process it in Python, and save a *new* file.
- **Metadata:** Always include a text file explaining column units (m, mm, kg/ha).
- **Anonymity:** If working with farmer data, remove Names and Phone Numbers before publishing code.

Chapter 4

Professional Resources

4.1 Recommended Libraries

As you advance, explore these specialized libraries:

- **PyTorch / TensorFlow:** For Deep Learning and Neural Networks.
- **PCSE (Python Crop Simulation Environment):** For crop modeling (WOFOST).
- **OpenCV-Contrib:** For advanced AR markers (ArUco) used in robotics.
- **Streamlit:** For turning your Python scripts into interactive Web Apps instantly.

4.2 Online Communities

- **Stack Overflow:** For debugging errors.
- **GitHub:** For finding open-source agricultural projects.
- **ArXiv (cs.CV):** For reading the latest Computer Vision papers.

End of Course Compendium

"The best way to predict the future is to invent it."

Electronics and Precision Agriculture Lab
Sokoine University of Agriculture